



MIR250 API REST + CONEXIÓN NODE- RED

Jon Ander Maiz - CF Somorrostro

Cómo acceder a REST API

- Para poder acceder, en el menú de la izquierda "?" → API Documentation.
- En la página que nos muestra, ponemos usuario y contraseña.
- Esto crea un "token" de autorización, que será el que utilizemos para hacerle las peticiones desde otro sitio.
- Accedemos pulsando en Open Documentation.

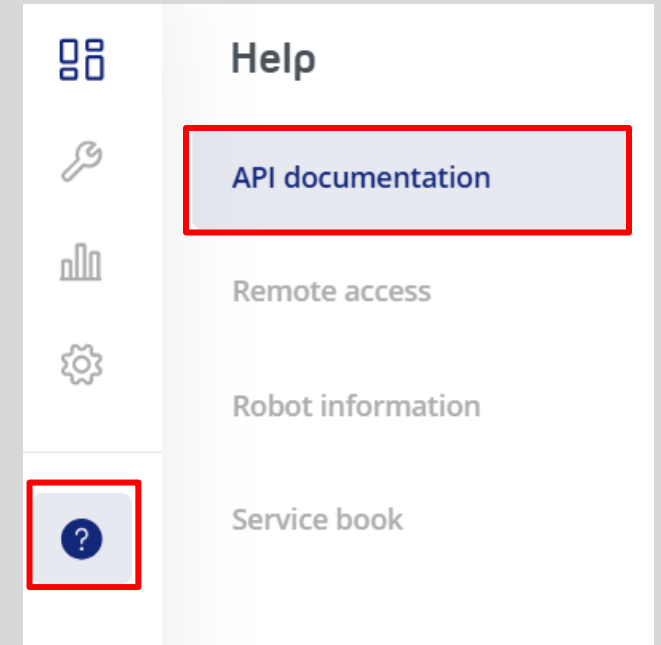
API credentials

Username
Enter your MiR username

User password
Enter your MiR user password
 

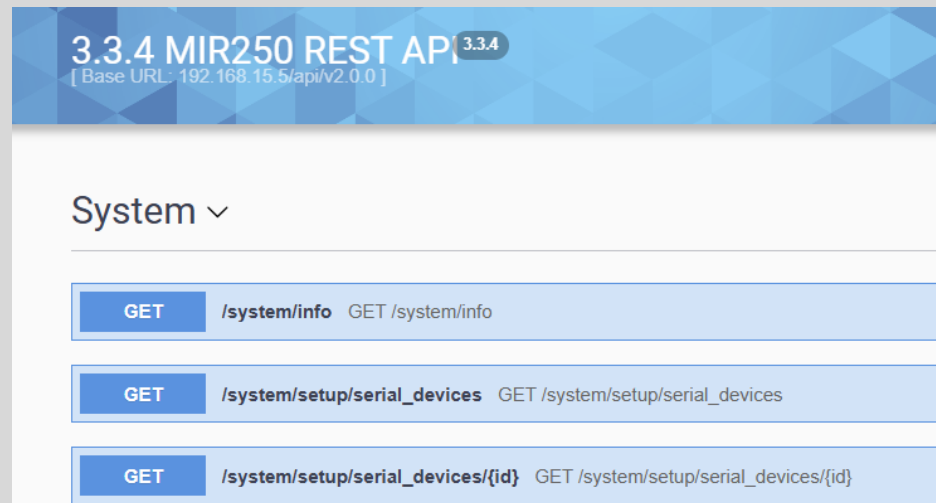
Open documentation

```
Content-Type : application/json
Accept-Language : en_US
Host : 192.168.15.5
Authorization : Basic ZGlzdHJpYnV0b3I6NjJm
```



¿Qué es una REST API?

- Es una **arquitectura de software** que define un conjunto de reglas (convenciones) para crear servicios web. Permite que diferentes sistemas (como una aplicación en Node-RED y el MIR 250) se comuniquen entre sí de manera **estandarizada, simple y sin estado** usando el protocolo HTTP/HTTPS, el mismo que usa tu navegador web.



Para que nos entendamos...

Vamos a ver la siguiente analogía

Un restaurante (API)

- **Tú (Cliente - Node-RED):** Eres el cliente que quiere un servicio.
- **La carta (Endpoints):** Es la lista de platos (operaciones) que puedes pedir.
- **El camarero (API REST):** Es el intermediario que toma tu pedido (solicitud) y lo lleva a la cocina.
- **La cocina (Servidor - MIR 250):** Es donde se prepara lo que pediste.
- **El plato que te sirven (Respuesta):** Son los datos (en formato JSON o XML) que recibes.

Operaciones basadas en métodos HTTP

Cada solicitud indica la acción que quiere realizar usando verbos estándar.

- **GET:** Obtener o leer datos (ej.: pedir el estado actual del robot).
- **POST:** Crear un nuevo recurso (ej.: iniciar una nueva misión).
- **PUT:** Actualizar un recurso existente (ej.: modificar un PLC Register).
- **DELETE:** Eliminar un recurso (ej.: cancelar una tarea programada).

Los datos (el recurso) se intercambian en un formato específico, típicamente **JSON** (el más común).

Ejemplo

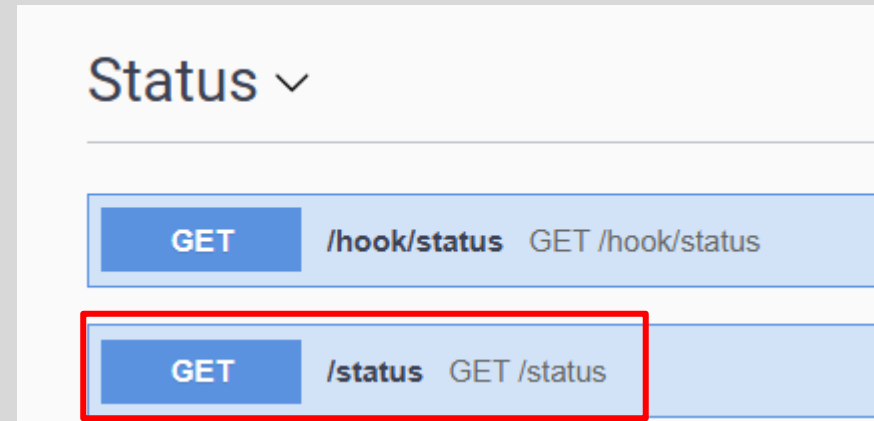
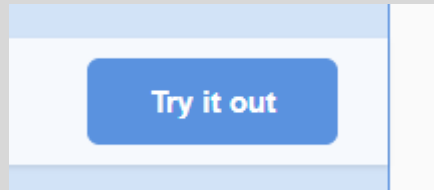
A través de la misma Rest API, podemos hacer pruebas para ver qué información nos devuelve.

- Vamos a pedir, que nos de el estado del robot.

Utilizando CTRL-F, buscamos la palabra Status, hasta llegar aquí.

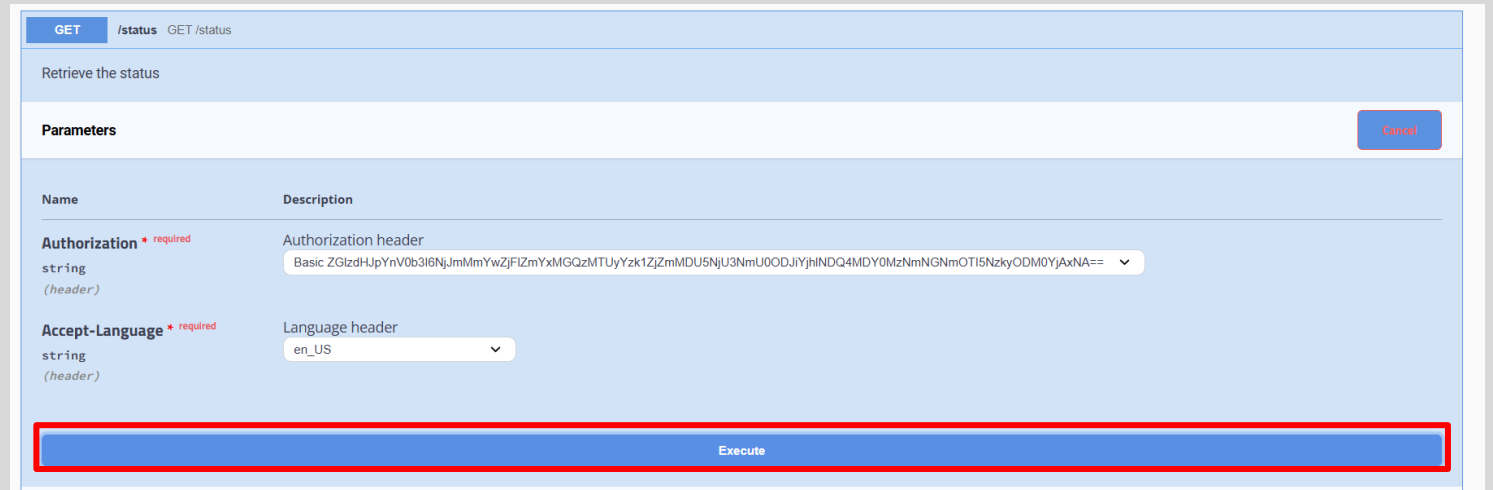
Haciendo click en cada parte, se abrirá como un desplegable.

Abrimos GET/status y le damos a



Ejemplo GET/status

- Se nos abre el siguiente desplegable. En este caso, no tenemos que escribir nada. Con darle a execute, podemos lanzar la petición



The screenshot shows a REST client interface for a GET request to /status. The request description is "Retrieve the status". Under the "Parameters" section, there are two required headers: "Authorization" with a value "Basic ZGZldHJpYnV0b3l6NjlmImYwZjFIZmYxMGQzMtUyYzk1ZjZmMDU5NjU3NmU0ODJiYjhlNDQ4MDY0MzNmNGNmOTI5NzkyODM0YjAxNA==" and "Accept-Language" with a value "en_US". A red rectangle highlights the "Execute" button at the bottom of the configuration panel.

Name	Description
Authorization * required string (header)	Authorization header Basic ZGZldHJpYnV0b3l6NjlmImYwZjFIZmYxMGQzMtUyYzk1ZjZmMDU5NjU3NmU0ODJiYjhlNDQ4MDY0MzNmNGNmOTI5NzkyODM0YjAxNA==
Accept-Language * required string (header)	Language header en_US

Execute

Ejemplo GET/status

- Lo primero que vemos, es la petición que ha lanzado.

Curl

```
curl -X GET "http://192.168.15.5/api/v2.0.0/status" -H "accept: application/json" -H "Authorization: Basic ZGZldHJpYnV0b3I6NjJmMmYwZjFlZmYxMGQzMtUyYzk1ZjZmMDU5NjU3NmU0ODJiYjhlNDQ4MDY0MzNmNGNmOTI5NzkyODM0YjAxNA==" -H "Accept-Language: en_US"
```

- De aquí sacaremos la estructura que tendrá nuestra petición en node-red.
- Lo siguiente que vemos es el código de respuesta y la respuesta (si todo ha ido bien).

Code	Details
200	Response body <pre>{ "position": { "x": 44.879852294921875, "y": 3.722496747970581, "orientation": 1.2323166131973267 }, "velocity": { "linear": 0, "angular": 0 }, "battery_time_remaining": 26127, "battery_percentage": 36.720001220703125, "moved": 2794.730565794026, "mission_queue_id": null, "mission_queue_url": null, }</pre>

Códigos de estado HTTP principales

- **2xx - Éxito:**
 - Ejemplo: **200 Ok** - Solicitud exitosa.
- **4xx - Error del 400 Bad Request: Scliente:**
 - Solicitud mal formada (ej.: JSON inválido).
- **5xx - Error del servidor:**
 - **500 Internal Server Error:** Error genérico del servidor (ej.: bug en el MIR 250).

Los más comunes en el contexto MIR 250 + Node-RED: 200 (éxito), 400 (petición incorrecta), 404 (ruta/robot no encontrado), 500 (error interno del robot).

Ejemplo POST/registers

- Vamos a buscar el siguiente endpoint.
- Le damos a Try it out, y se despliega la siguiente pestaña.

POST `/registers/{id}` POST /registers/{id}

id * required
integer
(path)

The id to add the new resource to

Register * required
(body)

The details of the register

Example Value | Model

```
{  
  "value": 23,  
  "label": "PLC Register 4"  
}
```

Registro PLC que queremos modificar

Valor que queremos darle

Nombre que queremos darle

*Si nos interesa cambiar solamente el valor, y no el nombre, podemos quitar el parámetro label.

Ejemplo POST/registers

- Para lanzar el comando → **Execute**
- Nos aparece debajo la respuesta del MIR.
- Y más abajo podemos ver las posibles respuestas.

En el panel de PLC registers del MIR250, podremos ver que se ha modificado el valor.

4

PLC Register 4

32

Code	Details
200 <i>Undocumented</i>	Response body <pre>{ "id": 4, "label": "PLC Register 4", "value": 32, "allowed_methods": ["PUT", "POST", "GET"] }</pre>

Node-red

- En node-red, tenemos que hacer la misma petición. Empezamos con el estado.
- Para ello vamos a hacer esta petición a un chatbot de nuestra elección.
 - Diseña un flujo en Node-RED que realice una petición **GET** al endpoint **/status** de una API REST (por ejemplo, de un robot MIR 250). Se conciso y no compliques el código, es para un ejemplo didáctico.
- Lo mínimo que vamos a necesitar, es un inject y el nodo http request.
- Después, necesitaremos un nodo para sacar/procesar la info, o por lo menos verla en el debug. El flujo básico será este.



Node-red

- El único nodo que tenemos que modificar aquí es "solicitud http".
- Método: GET/POST/PUT...
- URL: Dirección donde hacemos la petición.
- Devolver: Queremos un objeto JSON.
- Authorization: Basic XXXXX
- Content-Type: application/json

*La info está casi toda en las peticiones que hemos visto antes.

Curl

```
curl -X GET "http://192.168.15.5/api/v2.0.0/status" -H "accept: application/json" -H "Authorization: Basic ZG1zdHJpYnV0b3I6NjJmMmYwZjFlZmYxMGQzMtUyYzk1ZjZmMDU5NjU3NmU0ODJiVjh1NDQ4MDY0MzNmNGNmOTI5NzkyODM0YjAxNA==" -H "Accept-Language: en_US"
```

Editar nodo http request

Eliminar Cancelar Hecho

Propiedades

Método GET

URL http://192.168.15.5/api/v2.0.0/status

Carga Ignorar

☐ Habilitar conexión segura (SSL/TLS)

☐ Usar autenticación

☐ Habilitar conexión activa (keep-alive)

☐ Usar proxy

☐ Enviar solo respuestas que no sean 2xx al nodo Catch

☐ Deshabilitar el análisis estricto de HTTP

Devolver un objeto JSON analizado

Tip: If the JSON parse fails the fetched string is returned as-is.

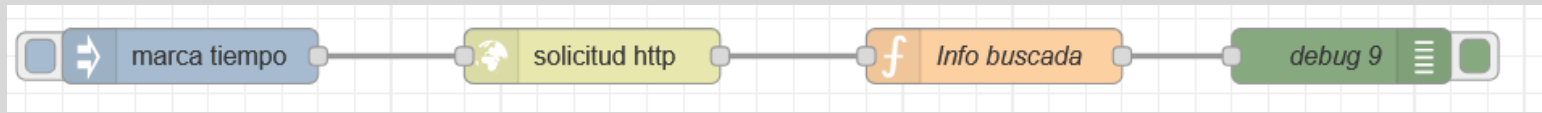
Encabezados

Authorization a_z

Content-Type application/json

Node-red - Status

- Instanciamos, y lanzando el flujo, vemos que nos devuelve mucha info sobre el estado del robot.
- Puede ser, que sólo necesitemos parte de esta información, para ello, pondremos una función después, que se quede con lo que buscamos.
- Vamos a suponer que sólo nos interesa el % de batería y el state_text.



Editar nodo function

Eliminar Cancelar Hecho

Propiedades

Nombre Info buscada

Configuración Al inicio En mensaje Al final

```
1 var bateria = msg.payload.battery_percentage
2 var texto_estado = msg.payload.state_text
3 msg.payload = {
4   "batería": bateria,
5   "estado": texto_estado
6 }
7 return msg;
```

msg.payload : Object

```
▼ object
  batería: 29.850000381469727
  estado: "Ready"
```

depuración

flujo actual todo

9/12/2025, 13:11:21 nodo: debug 9

msg.payload : Object

```
▼ object
  position: object
  velocity: object
  battery_time_remaining: 21369
  battery_percentage: 30.030000686645508
  moved: 2794.730565794026
  mission_queue_id: null
  mission_queue_url: null
  mission_text: "Waiting for new missions ..."
  distance_to_next_target: 0
  robot_name: "Dejavu"
  robot_model: "MiR250"
  serial_number: "206001015"
  session_id: "c689cd93-bea9-11ef-9d56-000e8eb69199"
  state_id: 3
  state_text: "Ready"
  mode_id: 7
  mode_text: "Mission"
```

Node-red – Modificar registro PLC

- En este caso, por delante del nodo de solicitud http, necesitaremos un nodo función donde le diremos el valor del registro PLC.
- El nodo debug sólo se pone para comprobar en node-red que la conexión es correcta y que se manda el dato.

The image shows a Node-RED workflow for modifying a PLC register. The flow consists of four nodes: a 'marca tiempo' (timer) node, a 'Valor a mandar' (function) node, a 'solicitud http' (HTTP request) node, and a 'debug 10' node. The function node is configured with the following JavaScript code:

```
1 msg.payload = {  
2   "value": 16  
3 };  
4 return msg;
```

The HTTP request node is configured with the following settings:

- Método: POST
- URL: `http://192.168.15.5/api/v2.0.0/registers/4`

Below the flow, a table displays the state of the PLC register:

4	PLC Register 4	16
---	----------------	----

On the right, a debug console shows the output of the 'debug 10' node:

```
9/12/2025, 13:19:11  nodo: debug 10  
msg.payload : Object  
  ▼ object  
    id: 4  
    label: "PLC Register 4"  
    value: 16  
    ▶ allowed_methods: array[3]
```

Node-red: herramienta de comunicación y programación

- Podemos llegar a lanzar misiones, parar misiones, comprobar parámetros del robot.
- Lo interesante de todo esto, es unirlo con el PLC, para poder comunicar en tiempo real toda la planta.
- En node-red podemos hacer distintas operaciones, ya que las funciones son sitios donde podemos programar lo que queramos.